factors that could make your solution cloud-ready within the ambit of open source technologies.

What is scalability?

What does the term scalability mean? Is it related to performance, availability, the reliability of the solution or is it related to cost management?

Scalability is the ability of an application to be able to scale to different parameters like:

- The ability to store large amounts of data when it needs
- The ability to perform a large number of transactions
- The ability to handle huge traffic surges

The new age tools, however, also consider cloud computing and help to reduce costs while keeping scalability in mind. They helps in optimising the costs when the solution reaches high scale thresholds.

Making scalable Web apps

Let's take a staged approach towards making a scalable Web application. Let us take a simple educational website that works like a portal, hosting articles and videos uploaded by faculty members, meant to be read and viewed by students.

Imagine this solution as a mix of YouTube and SlideShare, along with search capabilities, where students and faculty can upload and consume the content.

The most important requirement is to provide a stable and scalable platform to deliver the content.

To summarise the important requirements, the application should have the following:

1. The ability to store documents and videos of different sizes and even larger files up to a few GBs.
2. The ability to query and view the documents/ videos – there must be a player to render/play the documents or videos.
3. There ought to be no limit on the number of documents or videos the Web application can store.
4. There needs to be low latency for documents or videos to download.
5. It should be cost-effective.

We'll take the approach of a simple Web application, with a front-end and a back-end. Let's explore situations that may arise, and scale our application by using certain tools that will result in inherent changes to the app.

The deployment architecture of our Web application is shown in Figure 1.

The application has the following tiers:

1. The UI tier, which will have the presentation logic.
2. The Web API tier, which allows the application to integrate with third parties or app interfaces. This is especially important to integrate with mobile applications, third party content providers or external websites, as it allows maximising the reach of the application's content.
3. The business logic will consist of all the validation and logic for data processing. It is the soul of the entire application and all of the above tiers are dependent on this one.
4. The background jobs will perform all the heavy duty tasks without impacting the mainstream application's performance. In our example, whenever a user uploads the content, there can be many other background tasks that do the batch jobs like generating previews, creating thumbnails, indexing documents, sending notifications, emails, etc.
5. In order to streamline the load pertaining to certain requests, one can use queues (asynchronous mode), e.g., the process of printing the course completion certificates for students could be a good candidate for queues, wherein all jobs that need to be printed are in queue and the program can pick one after another to print the certificates.

Figure 2 shows the typical solution architecture of the Web application.

We will put down a few parameters while addressing the scalability of this solution. These are:

1. Ensuring that the application can handle the traffic surges.
2. Ensuring that the heavy duty processing caused when uploading content will not impact the experience of the content consumer.
3. Ensuring there is no adverse impact on existing logged-in users when adding an additional server to handle more load.
4. Fulfilling the requirement of storing large amounts of data (documents or videos).
5. Addressing low latency when downloading videos/documents.
6. Tackling cost savings while scaling out.
7. The background jobs will be working in a multi-threaded environment so one has to ensure that all such jobs are completed on time even with huge amounts of data to be processed.

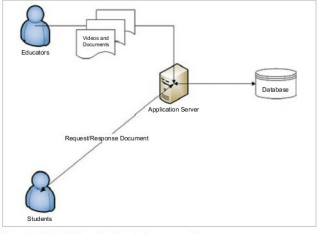

Figure 1: Typical Web application deployment architecture

Figure 2: Solution architecture of the Web application